Emerging Memory Technology: Surveying RRAM, HMC, and MemScale Architectures

1st Dominik Kornak
Electrical and Computer Engineering
University of Illinois at Chicago
Chicago, IL, USA
kornakdominik@gmail.com

2nd Adam Krzysztofek
Electrical and Computer Engineering
University of Illinois at Chicago
Chicago, IL, USA
adamkrzysztofek2005@gmail.com

3rd Michael Stanley
Electrical and Computer Engineering
University of Illinois at Chicago
Chicago, IL, USA
mas34@uic.edu

***Abstract*—In the last few decades, computing systems and their workloads have become substantially more memory intensive. As other components of our systems become more efficient and better equipped to deal with more demanding workloads, the need to improve our memory technology becomes increasingly important to avoid bottlenecking our systems. Currently, the industry standard for memory is dynamic random access memory (DRAM), which has been the primary form of main memory for decades. However, DRAM is increasingly unable to meet the demands of modern workloads.**

This survey highlights 3 specific technologies for improving or replacing DRAM, Metal-Oxide Resistive RAM (RRAM), Hybrid Memory Cube (HMC), and MemScale. RRAM offers an alternative to DRAM with the added benefit of non-volatility, lower power consumption, faster switching speeds, and increased density. HMC is a 3D-stacked memory architecture designed to significantly increase memory bandwidth through internal parallelism. MemScale introduces active low-power modes for memory systems and dynamically adapts the memory system to match memory demand.

This survey examines the techniques, performance, and trade-offs of these technologies, comparing their strengths and limitations. By analyzing their ability to overcome the shortcomings of DRAM, this work aims to provide insight into the future of memory system design and the role that these emerging technologies will play in next-generation computing systems.

I. Introduction

Modern Machine Learning algorithms and HPC (High Performance Computing) require more data intensive workloads and consume tons of power. Due to this, traditional memory architectures have become a limiting factor in overall system efficiency. Despite improvements in processor design, the gap between computational speed and memory access known as the memory wall has decreased the overall performance and capabilities of these new and innovating workloads. The main issue that engineers target is DRAM, which has been the primary form of main memory used by industry for many decades. DRAM is increasingly unable to keep up with the demands presented by these workloads: It is not large enough, faces severe scaling challenges, consumes too much power, and reduces performance due to memory interconnect latencies.

Our modern applications are rapidly constrained by these factors, typically termed the *Von Neumann bottleneck* or memory wall–leading, to an influx in research on emerging memory technologies. These innovations not only aim to replace existing memory technologies, but also to improve existing ones. As a result, solutions have emerged across the whole memory spectrum, from physical memory redesign, to architectural restructuring, and even software efficiencies.

This survey focuses on three different innovations: Resistive RAM (RRAM), the Hybrid Memory Cube (HMC), and MemScale. RRAM offers a promising non-volatile alternative to DRAM. Instead of storing data as charge on a capacitor as in the case of DRAM, RRAM exploits the resistive switching phenomenon observed in some materials to store data as either a high or low resistance state. This method is intrinsically non-volatile, thus circumventing the power hungry refresh operation required by DRAM, while also having higher density. However, its practical adoption is hindered by variability, endurance, and scaling issues arising from the stochastic nature of the switching process. The Hybrid Memory Cube is a 3D stacked memory architecture that significantly increases the memory bandwidth due to parallelization and also implements near memory computation. Finally, MemScale addresses system level energy efficiency by dynamically adjusting the clock frequency and voltage to match the workload demand.

By surveying these three approaches, we highlight the performance, speed, efficiency, and scalability of future memory systems. The goal of this survey is to provide and project the future in memory technology design.

II. Background and Related Work

Here, we provide a background on the underlying device physics governing the operation of RRAM, and how these processes lead to issues with device uniformity, endurance, and switching power in large scale RRAM arrays. In addition, we briefly discuss the history and structure of DRAM.

1) RRAM Fundamentals: Metal-Oxide RRAM exploits the resistive switching phenomenon observed in metal-insulator-metal structures to store data using the high resistance state (HRS) and low resistance state (LRS) of the devices, as depicted in Fig. 1. Initially, the device is in a very high resistance state. A forming voltage is applied to grow conductive filaments (CFs) from oxygen vacancy defects in the dielectric layer. After forming, RRAM cells can be cycled between LRS and HRS by applying SET and RESET voltages across the

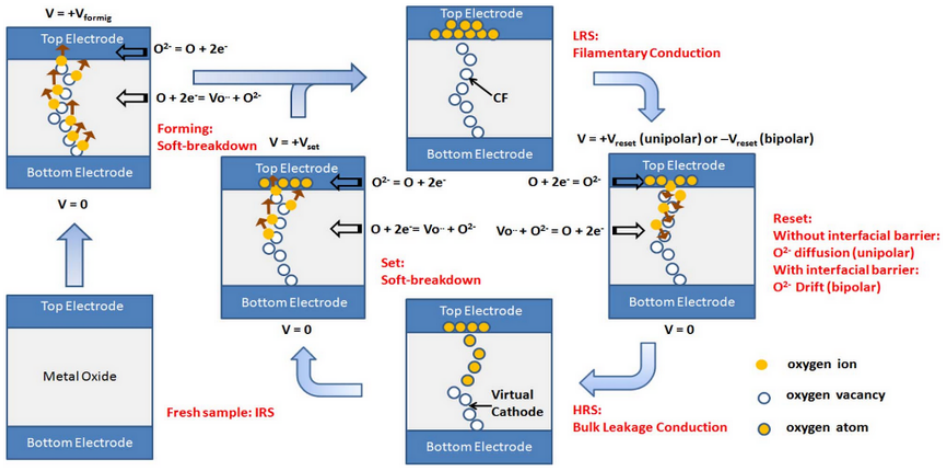

Fig. 1. Diagram of resistive switching stages in RRAM devices.

device. These voltages rupture (RESET) or reform (SET) the CFs. RRAM can be used in processing in memory schemes, consumes low power, and has higher density than DRAM. While RRAM has desirable properties, many challenges arise from the intrinsically stochastic nature of the CFs.

This stochasticity stems from the CF forming process. CFs form and rupture differently in each device, both in number and size. This process leads to inter-device uniformity problems in the resistance values of the LRS and HRS. Filament formation and rupture also varies from cycle to cycle intra-device, causing variation of the LRS and HRS states of each RRAM cell. Furthermore, the variability in the forming process affects switching voltages and endurance, complicating reliable and energy-efficient operation of RRAM at scale.

Many approaches have been proposed in an attempt to alleviate these issues. At the device level, uniformity is improved through advanced manufacturing techniques that focus on the electrode/oxide interface [2], [3] and methods that confine the CF formation paths [4]. Algorithmic approaches such as multiple voltage pulses for the SET and RESET operations have also been demonstrated to improve uniformity [5]. A verified RESET operation can be utilized to increase the endurance of RRAM cells, which essentially pushes oxygen back into the dielectric region. Scaling of RRAM suffers from the peak current that must be used during the reset process, which can be reduced by using a smaller set compliance current.

2) Existing Memory Architectures: As stated before, current memory architectures like DRAM do not offer enough performance for the applications that are preformed today. DRAM, or Dynamic Random Access Memory, has been the main volatile memory technology for decades due to its low production cost and high chip density. However, it does not meet today's standards due to its low bandwidth and high latency standards. DRAM technology has evolved over time, with increasing energy efficiency and higher bandwidth metrics. For example, DDR4 RAM increased the data rates to 3200 MT/s while reducing the operating voltage, which increased both energy efficiency and bandwidth [11]. Currently, DDR5 is the newest technology used for DRAM and it achieves data rates around 4800 MT/s and implements more parallelization through channel architecture and DRAM bank architecture [12].

Moreover, DRAM performance is constrained by the structural architecture itself. Every time a memory access is requested, a sequence of operations have to be preformed such as column and row activations and also pre-charging, which increases the latency. As mentioned previously, newer DRAM technologies like DDR5 exploit more bank level parallelism, it still doesn't eclipse the native architecture DRAM possess. On the energy side, DRAM is inefficient not in the data transfer aspect, but by the row activation and refresh operations required to maintain data integrity.

Another novel architecture is SRAM or Static Random Access Memory. SRAM is a volatile memory used in modern computing systems in areas such as processor caches due to its extremely low latency and high bandwidth. However, it suffers from scalability and cost because it contains six transistors per cell while DRAM contains only 1 transistor and a capacitor per cell. Due to this, SRAM is significantly less dense than DRAM because of the cost and scalability factors that come with it.

Phase Change Memory (PCM) is another emerging memory technology that uses the properties of chalcogenide materials such as $Ge_2Sb_2Te_5$ (GST) [14]. PCM functions by switching the material between two physical states, an amorphous high resistance state and a crystalline low resistance state. These states are switched from one to another through controlled heating. PCM offers advantages such as non volatility, good scalability, and fast read access times compared to DRAM. PCM also provides better endurance than that of other emerging memory architectures. However, if PCM were to be implemented in modern systems there would be a few challenges

to overcome. PCM's write process requires an enormous amount of energy due to the need for heating, repeated phase transitions can cause material degradation over time, and the resistance drift in the amorphous state introduces variability.

Idle Low Power States is a technique used in traditional DRAM architectures to reduce energy consumption by transitioning memory devices into low power modes during periods of idleness. When the memory system encounters an idle period, certain components such as DRAM banks or channels can be placed into power down or self refresh states, which reduce standby power. Although this approach is somewhat effective in reducing energy usage, it relies heavily on the system being able to accurately identify periods of inactivity. These frequent transitions between active and low power states can introduce latency into the system, especially if the idle periods are short or unpredictable. Low Power States are most effective in workloads with long periods of inactivity, but offer limited benefits in memory intensive or dynamic systems where memory is active for long periods of time.

Static Frequency Scaling is another technique used in traditional DRAM architectures to reduce the power consumption of memory by decreasing the frequency of DRAM. The frequency becomes a fixed number at runtime, and does not adapt to changes in workloads. This creates a simple technique that reduces energy efficiency of the memory because it doesn't offer dynamic monitoring and frequent transitioning. However, the lack of adaptability causes inefficiencies in memory systems. In systems that require high bandwidth, static frequency scaling does decrease the power consumption but it also decreases the performance and increases the access latency, negatively impacting the system.

Due to these inefficiencies with current memory architectures, designers have looked to both change the existing memory architectures and to improve current ones. This survey covers both aspects, new architectures being RRAM and the HMC, and current DRAM improvement in MemScale.

III. Schemes

This section covers the different architectural approaches each technique takes, and how the authors of each approach simulate the technologies. It also covers how each approach handles the memory wall bottleneck and how it improves the energy efficiency of memory.

A. RRAM

Resistive Random Access Memory (RRAM) has emerged as an alternative to DRAM due to it's simplicity, low power consumption, and scalability. RRAM highlights a fundamentally different approach to data storage, focusing on resistance based switching within dielectric materials. However, the physical processes that govern the RRAM operation introduce challenges in reliability and variability.

1) Controlling Device Uniformity: A major source of variability in RRAM devices stem from the stochastic nature of the forming process. Approaches such as [2] focus on engineering the metal-insulator interface in an attempt to achieve more controllable filament formation and rupture. A thin film of IrO_2 is deposited between Pt and NiO in a Pt/NiO/Pt memristor, forming a Pt/IrO_2/NiO/IrO_2/Pt memristor. Observations show that the thin IrO_2 layer causes the NiO layer to have an increase in its crystalline structure. The authors theorize that the highly crystalline NiO may help to confine CF formation and rupture, reducing the cycle to cycle variability of the devices.

Similarly, [3] experiments with embedding thin Al layers between the metal electrodes and insulating layers of HfO_2 based memristors in an attempt to stabilize local CF growth. By modifying the metal-insulator interfaces, these approaches aim to reduce the inherent spacial randomness exhibited in CF formation, increasing uniformity in conductance distributions and forming voltages. Improved uniformity leads to lower write energies, as predictable switching leads to reduced voltage margins, and simplified periphery circuitry.

Beyond interface engineering, techniques such as in [4] propose methods to confine the growth of conductive filaments to localized regions within the dielectric layer.

In this approach, the cell size of a ZrO_x/HfO_x RRAM device is simply scaled down to an area of 50 nm, reducing the regions in which CFs can grow. By restricting the areas in which CFs can grow, these approaches decrease the effective variability of the filament formation and rupture process. This leads to an improvement in uniformity, and thus lowered switching current and energy consumption as described before.

Outside of material engineering, algorithmic techniques have been developed to mitigate RRAM variability. Work such as [5] introduces multi-pulse programming schemes, where SET and RESET operations are performed using multiple voltage pulses rather than a single pulse. The pulse train was suggested by the authors to minimize failure, due to the observed Poisson distribution of the switching power. It is theorized that the pulse train supplies the device with the required power to switch, without the potentially damaging surge of a single voltage pulse.

2) Endurance Enhancement Techniques: As RRAM devices are cycled, the magnitude of the HRS resistance tends to decrease. This leads to a common failure state of the memristor becoming stuck in the LRS state, and decreased HRS/LRS ratios. It is theorized that this failure mode is caused by too many defects accumulating within the dielectric material as it is cycled.

Works such as [6] attempt to reverse this phenomenon using a verified RESET operation. In this process, a higher V_{RESET} (1.8 V instead of 1.5) is used to tighten the HRS distribution, and rehabilitate some devices with soft errors. Physically, the higher V_{RESET} effectively returns oxygen to the defect saturated dielectric region. By maintaining stable switching behavior across cycles, such techniques reduce device and write failures, simplifying redundant circuitry, and thus lowering energy consumption.

3) Scaling and Current Reduction: Scaling RRAM arrays introduces new challenges, particularly with respect to the peak current needed during the RESET process, which domi-

nates the peak power consumption. Problematically, it has been observed that the RESET current only slightly reduces as the device scales down, leading to a large current density needed for device RESET. Works such as [7] reduce the RESET current by observing that the RESET current is nearly linear with the SET compliance current. Therefore, reducing the SET compliance current reduces the RESET current, and thereby reduces the peak power consumption.

This section will cover the architectural layout of the Hybrid Memory Cube and how Memory is stored, transmitted, and managed within the structure. In addition, this section will cover the simulation setup, which integrates the HMC with a Kintex UltraScale Xilinx FPGA serving as the computational platform for issuing memory requests and collecting performance data. This setup enables detailed simulation and measurement of key metrics such as bandwidth, latency, and power consumption under different access patterns.

B. HMC

1) Memory Architecture: The Hybrid Memory Cube (HMC) strays away from conventional planar memory design by using three dimensional integration, rather than focusing on individual memory cells, HMC redesigns how memory is organized, accessed, and interfaced with the rest of the system. This architectural shift allows for increased bandwidth and increased energy efficiency; however, it also introduces new components and design considerations that must be understood.

First, we must understand how the HMC is actually structured and organized. As shown in Fig. 2, the HMC is a 3D stacked architecture where a logic layer is the base of the structure (blue) and DRAM is stacked in layers on top of it (gray). The 3D stack is then divided into 16 vertical vaults [9], where each vault gets its own memory controller. Each vault layer also contains 2 banks shown in red. Finally, these layers are connected by through-silicon vias (TSVs) which are vertically placed in each vault acting as the memory bus, so all DRAM

layers are connected meaning each memory controller is responsible for 8 DRAM layers. For the HMC 1.1 generation, each DRAM layer contains .5 GB of data, so for 8 layers of DRAM the total memory capacity is 4 GB. For newer generations like HMC 2.0, there are twice as many vaults, so there is twice as much data capacity (8 GB) [8].

A quadrant, shown in red in Fig. 2, is a group of four vaults. Each Quadrant gets an individual link used for external communication, and each link is capable of a bandwidth of 15 Gb/s. This means for a 4 link system with 8 lanes per link and full duplex, the maximum achievable bandwidth is 120 GB/s.

The HMC utilizes a closed page DRAM policy which means after a memory row is accessed, the word line is pre-charged and shutoff. This approach reduces leakage power when access are temporally spread out because it keeps rows shut off that will not be used. However, this implements latency for certain workloads when the access the same row subsequently, having the row shuts off increases the access time. This technique

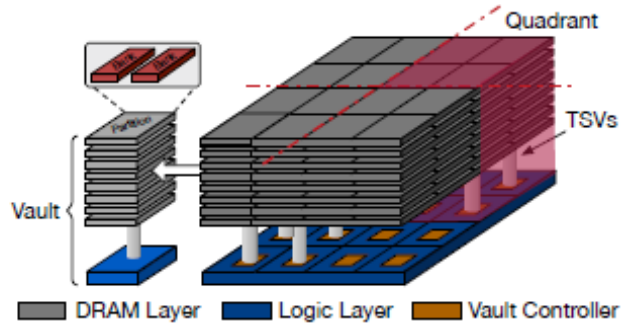

Fig. 2. HMC 1.1 Architecture.

increases the overall energy efficiency of the system while maintaining the high bandwidth parallel access.

Lastly, the HMC uses three different memory request types. The first request type is *ro* or Read-Only which reads data from memory, sending it to the processor. The next request type is *wo* or Write-Only, which writes to a specific location in memory. The last request type is *rw* or Read-Modify-Write which reads data from memory, sends it to its independent logic controller for computation, and then writes back to the memory location. This request utilizes the on-chip logic layer, and also utilizes the bi-directional link, increasing the serialization which improves performance. The *rw* request type reduces the need for data transfer between memory and the processor, eliminating link energy usage and also processor scheduling, increasing the efficiency.

2) HMC Communication: Also, the HMC utilizes a packet based communication protocol for memory movement. This technique provides higher bandwidth than older DRAM architectures because of there bus-based interfaces. The HMC packets are partitioned into 16 byte elements or otherwise known as flits. The actual packet sizes vary from 16 bytes to 128 bytes. The overhead in this architecture comes from the head and tail of the packet being a total of 16 bytes long regardless of the size. These header and tail additions are used to maintain packet integrity. To connect a single HMC to a processor or other memory, the Hybrid Memory Cube uses external links; either 2 or 4 of them. As mentioned before, each quadrant in the HMC gets an individual external link which improves the parallelization of the memory architecture. If one vault is accessed in a quadrant, another quadrants memory can be accessed without latency decreasing. This architecture improves the parallelization of the memory increasing the bandwidth, however it suffers form some setbacks. Firstly, the packet overhead causes routing delays which increase the HMC latency. Secondly, the difference in packet sizes causes variability in the performance of the system, which has to be optimized for.

3) HMC Simulation Setup: To characterize and test the HMC, it has to be incorporated and used in a system, not individually. As mentioned before, the authors use a Kintex UltraScale Xilinx FPGA with a 4GB HMC and connect this to a host system that controls the HMC and FPGA via PCIe

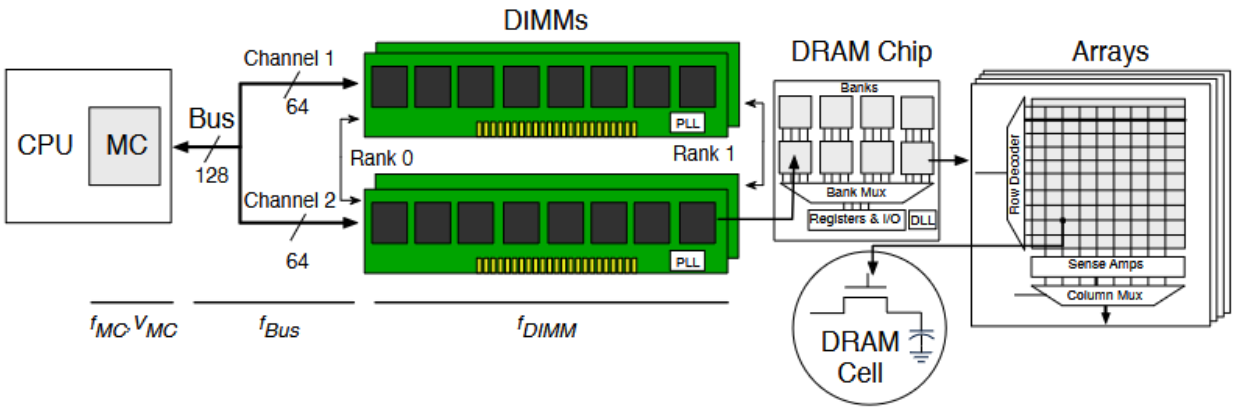

Fig. 3. DRAM Architecture.

3.0 x8 buses [1]. Since the authors use a 2 link system not 4, the maximum bandwidth is 60 GB/s between the FPGA and HMC [9]. This, however, is the bidirectional bandwidth, so the real one directional peak bandwidth the HMC can obtain in this setup is 30 GB/s.

To perform the simulation, the PICO API is used to initialize the hardware and establish communication between the host and the FPGA and a Verilog implementation of GUPS is used to generate memory requests to utilize the maximum given bandwidth of the system. GUPS, or Giga Updates Per Second, is a benchmark used to determine the ability memory handles random access. GUPS is implemented into this experiment in three different ways: Full Scale GUPS to maximize the bandwidth and monitor thermal behavior, Small Scale GUPS to adjust number of active ports, and stream GUPS which uses a streaming interface to provide efficient and quick communication between the host and the FPGA [8].

Due to the HMC's near memory computation, it not only improves the bandwidth of the memory architecture but improves the energy efficiency. Since the logic layer is implemented on the HMC chip, it reduces the amount of computation sent to the processor, which gives the processor less to do, increasing performance and decreasing the power needed. The logic layer does introduce some additional power consumption, however, the comparison between the logic layer and scheduling computation on the CPU is massive. In addition, the power needed to transfer data from memory to the CPU is mitigated with the on chip logic controller, saving additional power consumption.

C. MemScale

1) Memory Subsystem Organization: Unlike approaches that focus on redesigning memory devices or architectures, MemScale [10] targets system level energy efficiency by dynamically adapting memory operation based on workload demands. Instead of operating at a fixed voltage and frequency, MemScale controls memory parameters as the workload is being executed, allowing for continuous adaptation of the memory operation. MemScale aims to reduce energy consumption while maintaining performance.

Before diving into the specific techniques and mechanisms that MemScale employs, it is helpful to first understand some key background on modern memory subsystems.

Fig. 3 shows the different levels used in a memory subsystem. The memory controller (MC) sends instructions to the DIMMs, specifically the DRAM chips, over a memory bus. As illustrated in Fig. 3, processors have begun to include the MC within the processor itself, rather than as a separate component of the memory system. As shown, the width of the memory bus is divided into separate, independent channels to allow for increased parallelism.

Each DIMM contains register devices used to buffer incoming signals, a Phase-Lock Loop (PLL) and a chip level Delay Lock Loop (DLL) that control the device's frequency and phase, and multiple DRAM chips that provide data storage.

Fig. 3 also highlights the parameters that MemScale controls with DVFS and DFS. The parameters V_{MC} and f_{MC} control the voltage and frequency of the memory controller. Similarly, f_{Bus} and f_{DIMM} represent the memory bus and DIMM frequencies.

2) MemScale Goal: The main idea behind MemScale is to decrease the energy used by memory through dynamic adjustments in performance based on hardware counters, and instead of relying on idle periods, MemScale allows the memory system to operate in lower-power modes even while active.

MemScale achieves this by enabling the runtime adjustment of memory system parameters. In particular, it applies dynamic voltage and frequency scaling (DVFS) to the memory controller and dynamic frequency scaling (DFS) to the memory channels and DRAM devices. These components are scaled together during a workload, allowing the system to move between different power to performance operating points.

3) DVFS Background: Dynamic voltage and frequency scaling (DVFS) is not a new concept and has been widely used; it has been implemented in both modern CPUs and

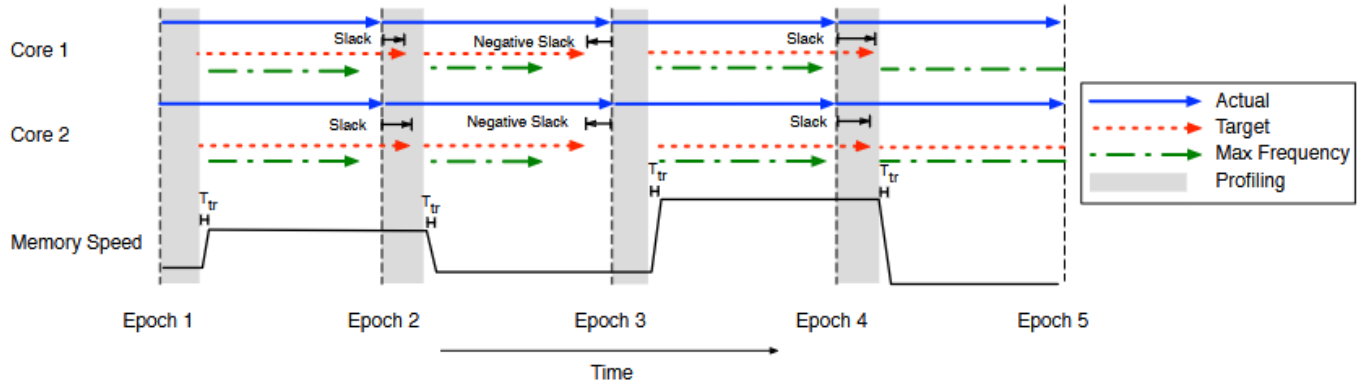

Fig. 4. MemScale Operation.

GPUs. In processors, DVFS is normally used to adjust core voltage and clock frequency based on workload intensity, reducing power consumption during low utilization while maintaining high performance when demand increases. This technique has been proven to be highly effective in balancing energy efficiency and performance. MemScale extends this well established technique to the memory system, allowing for similar opportunities in energy savings in memory bound workloads.

4) Performance Counter Monitoring: MemScale relies on performance counter monitoring to decide when to scale the system. For example, instruction counts and last-level cache misses are used to estimate how much execution time is spent waiting on memory, while additional counters track the number of outstanding memory requests at banks and channels to capture queueing delays. The system also monitors row buffer behavior, as well as row hits and misses, to estimate DRAM access latency. Together, these metrics allow the system to build a more complete view of memory demand than relying on a single indicator.

5) Performance Slack and Epoch Based Operation: A key concept used in the MemScale approach is performance slack, which represents how much slowdown is allowed compared to using the maximum frequency. MemScale determines a target performance based on a defined allowed slowdown threshold and compares it to the actual execution during each time frame, also called an epoch. If the system performs better than the target, it has positive slack and can reduce the memory frequency to save energy. However, if performance falls below the target, the system increases the frequency to recover performance.

MemScale operates in epochs as shown in Fig. 4, which begin by performing a profiling phase, where performance counters are collected to capture information about the current

workload. Next, the control algorithm uses this profiling information to determine a new memory frequency. The system then transitions to this new frequency by temporarily halting memory accesses. Finally, at the end of the epoch, the system evaluates performance and updates its internal estimates. This continuous repeating process allows for continuous adjustments to changes in workload behavior.

When selecting a frequency, MemScale does not choose the lowest possible setting, it instead considers both energy and performance together. For example, running at a lower frequency reduces memory power; however, it can also increase execution time, which may increase overall system energy. Due to this, MemScale selects the frequency that minimizes total system energy while still hitting the performance threshold.

IV. Results

A. RRAM

The techniques described in section III demonstrate measurable improvements in the uniformity, reliability, and energy efficiency of RRAM devices at scale. Fig. 6 demonstrates the impact of interface engineering on the uniformity of memristive devices in [2]. Compared to the baseline Pt/NiO/Pt device, the Pt/IrO_2/NiO/IrO_2/Pt structure clearly exhibits a significant stabilization in both the SET and RESET transitions.

Furthermore, the thin Al layer method of [3] demonstrated a reduction in the standard deviation of forming voltages from 1.53 to 0.27 V, and a reduction of the relative fluctuation (standard deviation/mean value) from 70.7 to 39.7% in the HRS, and from 76.2 to 10.4% in LRS.

Similarly, scaled down devices of [4] showed a large improvement in uniformity over larger devices in cycle to cycle performance and inter-device uniformity. Fig. 5 shows the significant improvement in switching variance when applying the voltage pulse technique for the SET and RESET operations in [5].

Endurance enhancement techniques further increase the reliability and energy efficiency of RRAM. The verified RESET method of [6] exhibited devices with high device yield ($\approx$100%), and high cycling endurance ($>10^6$).

Finally, scaling-oriented techniques focus in part on reducing the maximum current during the RESET process, which dominates energy consumption in RRAM operation. Works such as [7] have been able to reduce the maximum RESET current below 100 μA (and as low as 25 μA in [7]).

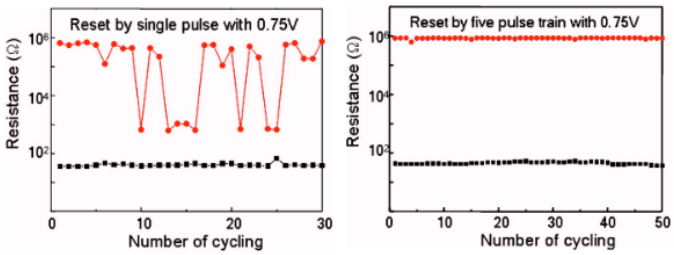

Fig. 5. Improvement in switching variance when using five successive pulses instead of one. Adapted from [5].

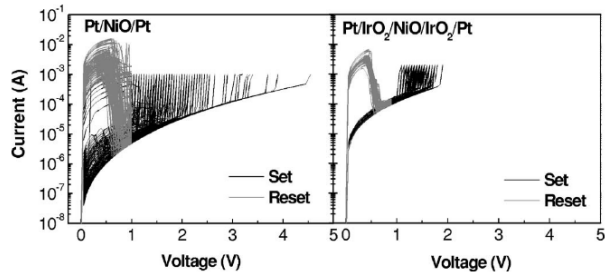

Fig. 6. Memristor switching IV curves over 200 cycles. The left figure is the Pt/NiO/Pt device, and the right is Pt/IrO_2/NiO/IrO_2/Pt. Gray lines are RESET transition, and black are SET. Adapted from [2].

B. HMC

With the simulation setup mentioned in the previous section, the authors simulate and monitor different parameters such as: Address Mapping, Bandwidth, Thermal, Power, and Latency.

The first simulation that was run was Address Mapping, which changes part of the access memory address to a set value so only a certain quadrant, vault, or bank, gets accessed during memory requests. This is achieved by forcing bits to 0 in the address so memory is accessed at a specific location. The results of this technique showed relative bandwidth trends across all memory requests, read only, write only, and read modify write. Since this bit masking forces only memory access at a specific location of the HMC, it is intuitive that the bandwidth will decrease. The lowest bandwidth shown in the simulation was when only one bank was used, which is the smallest independent memory unit in this architecture, which achieves a maximum bandwidth of about 4 GB/s, shown in Fig. 7. When the number of banks and vaults increases, the bandwidth increases, which shows how utilizing multiple banks/vaults in parallel increases bandwidth.

Since the maximum bandwidth of a vault is 10 GB/s, accessing more than 8 banks per vault does not increase the bandwidth due to link capacity. We see that the read modify write achieves the greatest bandwidth which is due to being 2 instructions bundled into one, read the data value and the write to it, which uses the bi-directional leak more efficiently. Since the request sizes for memory in the HMC can vary, it is important to determine which types of requests optimize the HMC. For a single request, the overhead is 16B regardless of the size. So, increasing the size of the request increases the overhead to data ratio. This improves the bandwidth per request since there is more data to be worked with, giving the HMC less overhead per request.

Since the HMC is a 3D stacked memory, it is prone to overheating since it is challenging to cool all layers effectively. Read only access is the only request type where the HMC does not overheat for the most basic cooling environment while write only and read modify write overheat with the same settings at the same access patterns. As shown in Fig. 7, the maximum temperature the HMC reaches is around 75 °C, which is its maximum operating temperature for write intensive applications. On the other hand, read only access has a higher operating temperature threshold of around 85 °C, which makes the HMC a better target for read intensive applications such as AI accelerators.

The power consumption of the HMC resembles the temperature pattern, when the number of banks and vaults increases, the temperature and power increase slightly. This is logical since increasing the number of vaults enlarges the active memory, which consumes more dynamic power.

As mentioned above, GUPS is used to create memory requests and monitor them. GUPS latency reading is not only RTT (Round Trip Time), but also the whole data packet setup and receival process. This adds overhead to the system which increases the latency to a best-case time of 547 ns; which is much longer than DDR4 competitors. Also, not only do smaller request sizes offer higher bandwidth capacity for multiple vaults, they also offer lower latency because there is less packet interference. For higher load memory access, utilizing more vaults and banks decreases the latency of the system while increasing performance. This is due to less concentration on a specific bank/vault which increases queuing latency and having less physical memory.

Comparing the HMC to DRAM, it actually introduces more latency due to its packet overhead. The HMC does not use a traditional DDR style memory interface, but relies on a packet switching architecture because of its design. Since it is split into multiple banks and vaults, packets are needed to address the independent sections and be scheduled efficiently.

Due to this, it adds additional overhead and potentially queuing delays, which adds additional latency. On average, the read latency for the HMC is around 192 ns [8], while DDR4 latency is around 15 ns. This drastic increase in latency is due to the packet structure overhead. However, the main metric considered is the bandwidth, with the HMC obtaining 160 GB/s to 320 GB/s for certain generations and configurations [8]. Compared to DRAM, DDR4 supports a 25 GB/s bandwidth [11] and DDR5 at a potential of 96 GB/s [12]. This shows that the HMC in bandwidth, which is the main topic of discussion when presented the problem of data movement in high performance applications.

Finally, the HMC may not look energy efficient but it certainly is, with DRAM achieving an approximate energy per bit of 3x compared to the HMC [13]. This means that DRAM approximately takes 3 times more energy to read or

write to a certain bit in memory. However, the HMC suffers from unnecessary power consumption when it has idle links, with the links

taking up significant amounts of power and not doing anything useful. Therefore, keeping the HMC running with all links used keeps it energy efficient. Potential optimizations for the HMC can include DVFS like MemScale mentioned in the next section, and also creating more efficient scheduling algorithms that best tailor to full use of the links.

C. MemScale

MemScale was evaluated using various simulations to best measure its effectiveness. At the beginning of the simulation, memory access traces were collected using the M5 simulator from a set of workloads running on a 16-core system. These workloads are composed of applications from the SPEC2000 and SPEC2006 benchmark suites and are grouped into three categories: memory-intensive (MEM), compute-intensive (ILP), and balanced (MID). In the second stage of the simulation, these traces are replayed using a memory system simulator that models the behavior of the operating system, the memory controller, and other required devices.

The simulator includes memory system behavior such as channel and bank contention, row buffer management, and power consumption. Furthermore, it also models the MemScale control policy. It includes performance monitoring, frequency scaling, and energy estimation. For comparison purposes, several baseline approaches are simulated and evaluated, including static frequency scaling, aggressive low-power states, and the Decoupled DIMMs approach.

The results show that MemScale is effective in reducing memory energy consumption while maintaining acceptable performance. Across all workloads, MemScale reduces memory energy consumption by approximately 17% to 71%. These savings also translate to an overall system energy reduction of about 6% to 31%. On average, MemScale hovers around 18.3% system energy savings with only about 4.2% performance degradation, while making sure that no application exceeds a maximum slowdown of roughly 9.2%.

The effectiveness of MemScale depends on the characteristics of the workload. Compute-intensive (ILP) workloads achieve the highest energy savings, with system energy reductions of up to 31%, as they do not require high memory bandwidth and can run at lower memory frequencies more often than not. Balanced (MID) workloads also show strong improvements, typically achieving at least 15% system energy savings. Memory-intensive (MEM) workloads see smaller gains, around 6% to 9%, because their higher memory demand limits opportunities for frequency scaling.

The results also show that MemScale dynamically adapts to changes in workload behavior over time. For example, during execution, the system can lower memory frequency during low-demand phases and increase it when memory demand rises. Even when there is a delay in detecting changes due to operating system scheduling intervals, MemScale is still able to maintain performance within the allowed limits.

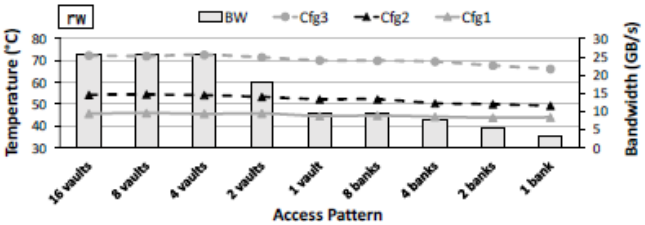

Fig. 7. Read Modify Write Temperature.

Compared to other approaches, MemScale consistently outperforms in terms of energy efficiency. Traditional power down techniques provide only small energy savings, typically under 7%, since they depend on idle periods. Static frequency scaling performs better, achieving around 19% system energy savings in some cases, but it cannot adapt to changing workload behavior. MemScale improves on this by dynamically adjusting memory frequency, achieving up to 30.2% system energy savings for compute-intensive workloads and around 16.9% for balanced workloads.

Further analysis shows that MemScale remains effective under various system configurations. Increasing memory traffic does, however, reduce the amount of energy savings. Similarly, varying parameters such as allowed performance degradation, number of memory channels, and workload specifics affect the results, but MemScale is able to maintain a balance between energy savings and performance. Overall, these results demonstrate that dynamic adaptation is key to improving memory energy efficiency.

Conclusion

In the ever evolving field of modern computing, the gap between processing power and memory performance remains a significant bottleneck in current systems. The increased demands of intensive data focused applications such as machine learning, high performance computing, and large scale data operations places a lot of pressure on memory subsystems. Although DRAM has been the backbone of main memory for decades, it is increasingly unable to meet the requirements of scalability, performance, and energy efficiency. This survey explored 3 different approaches to dealing with the said limitations of DRAM: Resistive RAM (RRAM), Hybrid Memory Cube (HMC), and MemScale. Each of these technologies targets a different layer of the memory hierarchy, from architectural changes to system level optimization.

RRAM presents as a worthwhile alternative to DRAM by introducing nonvolatile memory with lower power consumption and faster switching speeds. However, RRAM faces several obstacles that must be addressed before it can replace DRAM, variability in conductive filament formation, endurance limitations from repeated switching, and scaling related current density issues, of which all are significant challenges. These problems can be mitigated to some extent through techniques surveyed, such as interface engineering, filament confinement, and multi pulse programming.

HMC introduces a fundamentally different approach to memory design by implementing 3D stacking and increased internal parallelism. HMS organizes memory into vertically stacked layers; this allows for significantly higher bandwidth compared to traditional 2D memory architectures. In an HMC, memory is divided into vaults, each with its own memory controller, to allow concurrent access patterns that improve throughput. As shown in the results, HMS is effective in bandwidth intensive workloads, where parallel access to multiple banks and vaults can be fully utilized. The 3D structure of an HMC does introduce thermal challenges; heat dissipation becomes more difficult due to the stacked layers. An HMC offers substantial improvements in performance; however, its deployment requires careful consideration of the constraints in the system.

MemScale focuses on improving the energy efficiency of memory, specifically DRAM, through dynamic scaling rather than replacing the underlying architecture. MemScale optimizes the operation of current memory systems through the application of dynamic voltage and frequency scaling (DVFS) and dynamic frequency scaling (DFS) based on the characteristics of a real time workload. MemScale continuously adapts by leveraging hardware performance counters and performance slack to minimize energy consumption while maintaining acceptable performance. The strength of MemScale comes from its ability to adapt to workload behavior over time, allowing it to be effective in workloads with varying memory demands.

A key takeaway from this survey is that no single solution is completely sufficient to address the challenges that current memory systems face. The future of memory technology will likely consist of a combination of innovations in each level of the memory hierarchy. Device level technologies such as RRAM offer improved energy efficiency and storage characteristics, while architectural level solutions like HMS address the growing demand of increased bandwidth and parallelism. System level techniques such as MemScale complement these approaches by ensuring the memory systems can operate and adapt efficiently when dealing with varying workloads. The next challenge will be to allow these various techniques and technologies to work together, allowing them to face continued increases in demands of memory performance.

Furthermore, these approaches are not mutually exclusive; each of these advances in technology could be modified to work together. For example, non volatile memory like RRAM could be integrated into 3D stacked architecture similar to HMC, resulting in both increased memory density and increased bandwidth while also applying techniques like MemScale to further optimize energy consumption.

In conclusion, the future of memory system design will depend on a combination of innovation. As workloads evolve and demand greater performance and efficiency, multiple approaches must be applied to keep up with current workload trends. A strategy that incorporates changes in designs, architectures, and systems will be required to keep up with demands. The technologies explored in this survey: RRAM, HMC, and MemScale demonstrate the diverse and complementary directions in which memory research is developing. Together, they provide valuable information about how future memory systems will overcome current limitations and enable more efficient, scalable, and high performance computing platforms.

References

[1] Wong, H-S. Philip, et al. "Metal–oxide RRAM." Proceedings of the IEEE 100.6 (2012): 1951-1970.

[2] Kim, D. C., et al. "Improvement of resistive memory switching in NiO using IrO2." Applied physics letters 88.23 (2006).

[3] Yu, Shimeng, et al. "Improved uniformity of resistive switching behaviors in HfO2 thin films with embedded Al layers." Electrochemical and Solid-State Letters 13.2 (2010): H36-H38.

[4] Lee, Joonmyoung, et al. "Diode-less nano-scale ZrO x/HfO x RRAM device with excellent switching uniformity and reliability for high-density cross-point memory applications." 2010 International Electron Devices Meeting. IEEE, 2010.

[5] Yoo, I. K., et al. "Interpretation of nanoscale conducting paths and their control in nickel oxide (NiO) thin films." Applied Physics Letters 92.20 (2008).

[6] Chen, Y. S., et al. "Highly scalable hafnium oxide memory with improvements of resistive distribution and read disturb immunity." 2009 IEEE International Electron Devices Meeting (IEDM). IEEE, 2009.

[7] Lee, Heng-Yuan, et al. "Low power and high speed bipolar switching with a thin reactive Ti buffer layer in robust HfO2 based RRAM." 2008 IEEE International Electron Devices Meeting. IEEE, 2008.

[8] R. Hadidi, B. Asgari, B. A. Mudassar, S. Mukhopadhyay, S. Yalamanchili, and H. Kim, "Demystifying the characteristics of 3D-stacked memories: A case study for Hybrid Memory Cube," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, 2017, pp. 66–75.

[9] T. Pawlowski, "Hybrid Memory Cube (HMC)," in *Proc. Hot Chips Symp. (HCS)*, 2011, pp. 1–24.

[10] Qingyuan Deng, David Meisner, Luiz Ramos, Thomas F. Wenisch, and Ricardo Bianchini. 2011. MemScale: Active Low-Power Modes for Main Memory. In Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems (Newport Beach, California, USA) (ASPLOS XVI). Association for Computing Machinery, New York, NY, USA, 225–238.

[11] JEDEC Solid State Technology Association, "DDR4 SDRAM Standard (JESD79-4)," 2012.

[12] JEDEC Solid State Technology Association, "DDR5 SDRAM Standard (JESD79-5)," 2020.

[13] Xun Jian, Pavan Kumar Hanumolu, and Rakesh Kumar. 2014. Understanding and Optimizing Power Consumption in Memory Networks. In Proceedings of the International Symposium on Computer Architecture (ISCA). IEEE/ACM.

[14] H.-S. Philip Wong, Simone Raoux, SangBum Kim, Jiale Liang, John P. Reifenberg, Bipin Rajendran, Mehdi Asheghi, and Kenneth E. Goodson. 2010. Phase Change Memory. Proceedings of the IEEE 98, 12 (2010), 2201–2227.